

MeatShop Threat Pipeline

How the two-server deployment is hardened, how traffic is balanced across the paired Spring Boot containers, and how a locally hosted Llama 3 reads the log stream on a fixed cadence and writes scored threat verdicts back into Grafana.

HOSTS	APP INSTANCES	BALANCING	
2 (app + analysis)	2 × Spring Boot 3.4.5	Nginx round-robin	
MODEL	DASHBOARD REFRESH	LOG RETENTION	
Llama 3 via Ollama	10 s	72 h (Loki)	

01	Executive summary	02	System architecture	03	Host hardening
04	Edge and load balancing	05	Application tier	06	Log transport
07	AI analysis server	08	Cycle timing algorithm	09	Threat simulation runbook
10	Known gaps				

Addresses, ports and credentials in this document are placeholders. Substitute your own before applying any configuration.

01 Executive summary

MeatShop runs on two machines with different jobs. The first carries public traffic and everything that stores state: Nginx at the edge, two identical Spring Boot containers behind it, PostgreSQL, and the Grafana observability stack. The second machine holds no application data at all. It exists to read logs, ask a local language model what they mean, and write the answer back as structured alerts.

Splitting the work this way keeps model inference away from request-serving CPU, and it keeps log content inside the perimeter. Llama 3 runs

under Ollama on the analysis host, so no log line is ever sent to a third-party inference API.

AVAILABILITY

N+1

Either backend container can be stopped for a deploy without dropping requests. Nginx marks a dead upstream after 3 failures and retries the other.

EXPOSURE

2 ports

Only 443 and the relocated SSH port face the internet. Postgres, Loki, Grafana and both app containers bind to the loopback or the private link.

DETECTION LAG

≤ 5 min

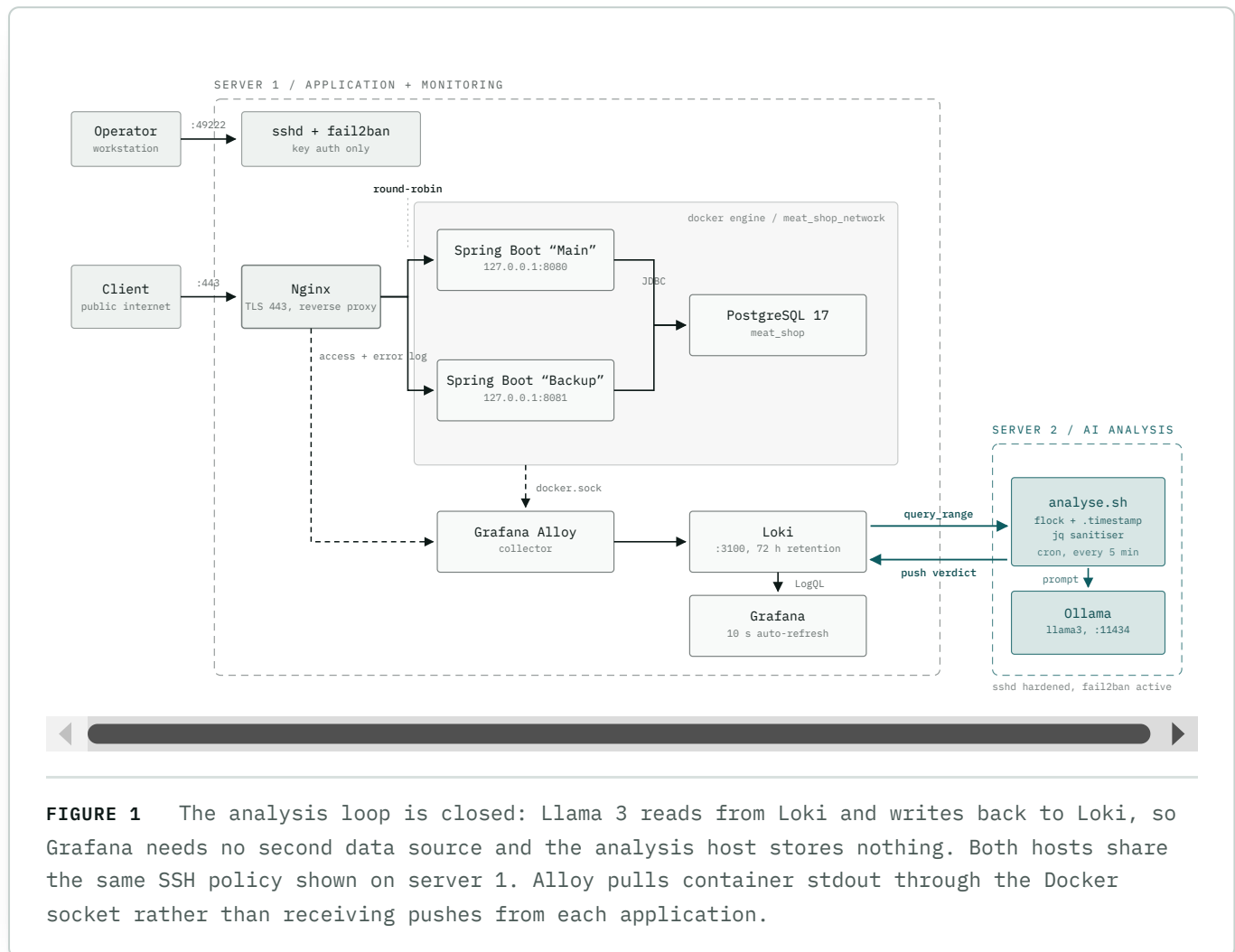
Worst case from a log line landing in Loki to a scored verdict appearing on the dashboard, assuming a 5-minute analysis interval.

What each layer contributes

RESPONSIBILITY SPLIT ACROSS THE TWO HOSTS		
COMPONENT	HOST	JOB IN THE DETECTION PATH
sshd + Fail2ban	both	Blocks credential-guessing at the network layer before it reaches an application log at all.
Nginx	server 1	Terminates TLS, spreads requests across both app instances, and writes the access log that Fail2ban and Alloy both read.
Spring Boot × 2	server 1	Emits structured JSON log events carrying requestId, ip, endpoint and instanceName.
PostgreSQL	server 1	Emits statement and connection errors that reveal injection attempts and authentication probing.
Grafana Alloy	server 1	Tails container stdout and the Nginx log files, labels each stream, ships to Loki.
Loki	server 1	Single queryable store for every log stream, and also the destination for the AI verdicts.
Ollama + Llama 3	server 2	Reads a time-bounded slice of logs and returns a risk score with a stated reason.
Grafana	server 1	Renders the risk gauge, threat table and raw payload log, refreshing every 10 seconds.

02 System architecture

The drawing below is the whole system on one plate. Solid lines carry request traffic, dashed lines carry telemetry, and the teal path is the analysis loop that closes back on itself through Loki.



Request path, end to end

1. A client opens TLS to Nginx on 443. Nginx is the only process bound to a public interface besides `sshd`.
2. Nginx picks the next upstream in rotation and proxies to `127.0.0.1:8080` or `127.0.0.1:8081`, adding `X-Real-IP` and `X-Forwarded-For`.
3. The Spring Boot instance handles the request. `RequestLoggingFilter` stamps a UUID `requestId` into the MDC along with the client IP, method, endpoint and `instanceName`, so every line from that request is correlatable.

4. Business queries hit PostgreSQL over the private Docker network. Nothing else reaches the database.
5. Logback writes a JSON event per line to container stdout. Alloy reads it off the Docker socket, attaches `service`, `level` and `application` labels, and pushes to Loki.
6. Every 5 minutes the analysis host asks Loki for the slice of log lines it has not read yet, sends them to Llama 3, and pushes a scored verdict back under a dedicated job label.
7. Grafana, refreshing on a 10-second interval, unwraps `risk_percentage` from those verdicts into the gauge and threat table.

03 Host hardening

Both machines are configured identically at the OS level. The goal is narrow: remove password guessing as an attack surface entirely, then rate-limit whatever is left.

SSH daemon

The listener is moved off 22 and password authentication is switched off, not merely discouraged. Moving the port does not stop a determined scanner, but it removes the constant background noise of untargeted bots, which makes the Fail2ban log meaningful instead of saturated.

/etc/ssh/sshd_config

SERVER 1 AND SERVER 2

```
# --- listener -----
Port 49222
AddressFamily inet
ListenAddress 0.0.0.0

# --- authentication -----
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
PasswordAuthentication no
PermitEmptyPasswords no
ChallengeResponseAuthentication no
KbdInteractiveAuthentication no
UsePAM yes
```

```
PermitRootLogin no
AllowUsers deploy ops

# --- session limits -----
MaxAuthTries 3
MaxSessions 4
LoginGraceTime 20
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no

# --- modern crypto only -----
KexAlgorithms curve25519-sha256,curve25519-sha256@libssh.org
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com
LogLevel VERBOSE
```

ORDER OF OPERATIONS

Open the new port in the firewall and confirm a working key login on a second terminal *before* reloading `sshd`. Applying `PasswordAuthentication no` and a port change together with no verified key in place locks the host out. `sshd -t` validates the file; `systemctl reload ssh` keeps existing sessions alive.

firewall

UFW, APPLIED ON BOTH HOSTS

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 49222/tcp          # ssh, relocated
ufw allow 443/tcp           # nginx, server 1 only
ufw allow from 10.10.0.12 to any port 3100 proto tcp # loki, analysis host only
ufw enable
```

Fail2ban

Three jails run. The first covers SSH itself. The second and third read the Nginx access log, which is the earliest place an application-level brute force becomes visible with a source IP attached. Bans are written to the firewall, so a banned address stops consuming worker threads.

JAIL THRESHOLDS

JAIL	WATCHES	TRIGGER	BAN
sshd	journalld, sshd unit	4 fails / 10 min	2 h
meatshop-auth	Nginx access log, POST /auth/login	8 fails / 5 min	1 h
meatshop-probe	Nginx access log, 4xx scanning	30 hits / 2 min	30 min

/etc/fail2ban/jail.local	SERVER 1
<pre>[DEFAULT] backend = systemd banaction = ufw ignoreip = 127.0.0.1/8 10.10.0.0/24 findtime = 600 bantime = 7200 maxretry = 4 [sshd] enabled = true port = 49222 logpath = %(sshd_log)s [meatshop-auth] enabled = true port = http,https filter = meatshop-auth logpath = /var/log/nginx/meat-shop-access.log findtime = 300 maxretry = 8 bantime = 3600 [meatshop-probe] enabled = true port = http,https filter = meatshop-probe logpath = /var/log/nginx/meat-shop-access.log findtime = 120 maxretry = 30 bantime = 1800</pre>	

/etc/fail2ban/filter.d/meatshop-auth.conf	LOGIN FAILURE FILTER

[Definition]

```
# MeatShop answers a bad password with 403 and an unknown email with 403,  
# so both land in the same jail. 401 covers an expired or forged token.  
failregex = ^<HOST> \S+ \S+ \[[^]]+\] "POST /auth/login HTTP/[\d.]+" (401|403)  
           ^<HOST> \S+ \S+ \[[^]]+\] "POST /auth/refresh HTTP/[\d.]+" (401|403)  
ignoreregex =  
datepattern = ^[^\[]*\[({DATE})
```

Validate the regex against real traffic before enabling the jail, otherwise a format mismatch produces a jail that never fires and gives false confidence:

verification

SHELL

```
fail2ban-regex /var/log/nginx/meat-shop-access.log \  
  /etc/fail2ban/filter.d/meatshop-auth.conf --print-all-matched  
  
fail2ban-client status meatshop-auth  
fail2ban-client set meatshop-auth unbanip 203.0.113.9
```

04 Edge layer and load balancing

Nginx does three jobs: TLS termination, round-robin distribution, and passive failover. Round-robin is what `upstream` does when no balancing directive is given, so the block below is round-robin precisely because it names no algorithm.

/etc/nginx/conf.d/meatshop.conf

SERVER 1

```
# Access log format. Fail2ban's HOST capture reads the first field,  
# so remote_addr has to stay in position one.  
log_format meatshop '$remote_addr - $remote_user [$time_local] '  
    '$request' $status $body_bytes_sent '  
    '$http_referer' '$http_user_agent' '  
    'upstream=$upstream_addr rt=$request_time urt=$upstream_response_t  
  
upstream meatshop_api {  
    # No directive here means round-robin. Adding least_conn or  
    # ip_hash would change the algorithm; both are deliberately absent.  
    server 127.0.0.1:8080 max_fails=3 fail_timeout=15s;  
    server 127.0.0.1:8081 max_fails=3 fail_timeout=15s;
```

```

    keepalive 32;
}

limit_req_zone $binary_remote_addr zone=authzone:10m rate=10r/m;

server {
    listen 443 ssl http2;
    server_name api.meat-shop.com;

    ssl_certificate      /etc/letsencrypt/live/api.meat-shop.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/api.meat-shop.com/privkey.pem;
    ssl_protocols        TLSv1.2 TLSv1.3;

    access_log /var/log/nginx/meat-shop-access.log meatshop;
    error_log  /var/log/nginx/meat-shop-error.log warn;

    server_tokens off;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

    # Login is rate limited harder than the rest of the API.
    location = /auth/login {
        limit_req zone=authzone burst=5 nodelay;
        proxy_pass http://meatshop_api;
        include /etc/nginx/snippets/meatshop-proxy.conf;
    }

    # Actuator stays internal.
    location /actuator/ {
        allow 10.10.0.0/24;
        deny all;
        proxy_pass http://meatshop_api;
        include /etc/nginx/snippets/meatshop-proxy.conf;
    }

    location / {
        proxy_pass http://meatshop_api;
        include /etc/nginx/snippets/meatshop-proxy.conf;
    }
}

```

/etc/nginx/snippets/meatshop-proxy.conf

SHARED PROXY BEHAVIOUR

```

proxy_http_version 1.1;
proxy_set_header Connection "";

```



```
proxy_set_header Host                $host;
proxy_set_header X-Real-IP           $remote_addr;
proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto  $scheme;

# Failover. If an upstream refuses, times out or answers 5xx,
# the request is replayed against the other instance.
proxy_next_upstream error timeout http_502 http_503 http_504;
proxy_next_upstream_tries 2;
proxy_next_upstream_timeout 8s;

proxy_connect_timeout 3s;
proxy_send_timeout    30s;
proxy_read_timeout    30s;
```

How failover actually behaves

Open-source Nginx has no active health checking. It does not poll `/actuator/health`. An instance is marked down only after live traffic fails against it, which is what `max_fails=3 fail_timeout=15s` encodes: three failures inside 15 seconds takes the upstream out of rotation for 15 seconds, then one probe request is allowed through to test it.

The practical effect during a rolling deploy is that a small number of requests hit the container as it shuts down. `proxy_next_upstream` catches most of those and replays them against the surviving instance, so the client sees a slower response rather than an error. Requests that are not idempotent are the exception, and `POST /auth/login` is the one worth watching.

CLIENT IP IS LOST WITHOUT ONE MORE SETTING

`RequestLoggingFilter` populates the MDC `ip` field from `request.getRemoteAddr()`. Behind a proxy that returns the Nginx address, so every log line records `127.0.0.1` and the AI analysis loses its most useful correlation key. Set `server.forward-headers-strategy: NATIVE` in `application.yaml` so Spring resolves `X-Forwarded-For` into the remote address. The `AuthController` already reads `X-Real-IP` directly, so login events survive either way, but nothing else does.

05 Application tier

Both containers run the same image, tagged from the repository `VERSION` file by the GitHub Actions pipeline. They differ only in two environment variables, and those two variables are what makes a log line attributable to an instance.

IMAGE

`aehamalhaeak/meat_shop:${BACKEND_VERSION}`, built from `BackEnd/Dockerfile` on `eclipse-temurin:21-jre-alpine`

INSTANCE 1

`meat_shop_backend`, `INSTANCE_ID=1`, `INSTANCE_NAME=Main`, published on 8080

INSTANCE 2

`meat_shop_backend_2`, `INSTANCE_ID=2`, `INSTANCE_NAME=Backup`, published on 8081

SHARED STATE

Both point at the same PostgreSQL container. Sessions live in the `sessions` table, not in memory, so either instance can serve any request without sticky routing.

SCHEMA

Flyway runs at startup with `baseline-on-migrate: true`. Hibernate is set to `ddl-auto: validate`, so a schema drift fails the boot rather than silently altering tables.

HEALTH

`/actuator/health` and `/actuator/prometheus` are permitted without a token in `SecurityConfig`, and restricted to the private range at the Nginx layer.

STATELESSNESS IS WHAT MAKES ROUND-ROBIN SAFE

Spring Security is configured `SessionCreationPolicy.STATELESS` and there is no servlet session to replicate. Authentication state is a row in `sessions` carrying the hashed refresh token, the device fingerprint baseline and a trust score. Instance 1 can issue a token that instance 2

validates on the next request, which is exactly what round-robin will produce.

Recommended compose changes for a proxied deployment

The committed compose file publishes both application ports on all interfaces and runs the `dev` profile. Behind Nginx, neither is right. Bind to loopback so the containers are unreachable from outside the host, and switch to `prod` so Logback emits JSON rather than the human-readable pattern.

Docker/docker-compose.yaml

EXCERPT, HARDENED

```
backend:
  image: aehamalhaeak/meat_shop:${BACKEND_VERSION}
  container_name: meat_shop_backend
  environment:
    SPRING_PROFILES_ACTIVE: prod          # JSON logs for machine reading
    SERVER_FORWARD_HEADERS_STRATEGY: NATIVE
    INSTANCE_ID: 1
    INSTANCE_NAME: Main
  ports:
    - "127.0.0.1:8080:8080"  # loopback only, nginx is the door
  depends_on:
    postgres: { condition: service_healthy }
  restart: unless-stopped
  logging:
    driver: json-file
    options: { max-size: "20m", max-file: "5" }

backend-2:
  # identical, with INSTANCE_ID 2, INSTANCE_NAME Backup, port 8081
```

TWO GAPS WORTH CLOSING BEFORE THIS COUNTS AS HA

Neither backend declares `restart: unless-stopped`, so a crashed container stays down until someone notices. And only `postgres` has a healthcheck, which means `docker compose up` reports success while an application is still starting. Add an application healthcheck against `/actuator/health` and the failover story stops depending on manual intervention.

06 Log transport into Loki

Alloy is the only component that touches raw logs. It discovers containers through the Docker socket, keeps the four that matter, extracts a severity label from each line, and forwards everything to Loki. The Nginx files are tailed separately because they are written to disk rather than to a container stream.

STREAMS AVAILABLE TO A LOGQL QUERY		
SELECTOR	SOURCE	WHAT IT CARRIES
{service="backend"}	container stdout	Login failures, authorization denials, session state transitions, unhandled exceptions
{service="backend_2"}	container stdout	Same, from the second instance
{service="postgres"}	container stdout	Statement errors, constraint violations, connection failures
{service="nginx"}	file tail	Request line, status, source IP, upstream that served it, response time
{job="meatshop_backend_ai_alerts"}	AI push	Scored verdicts about the backend streams
{job="meatshop_postgres_ai_alerts"}	AI push	Scored verdicts about the database stream

What a backend log line looks like

Under the `prod` profile, `logback-spring.xml` uses the Logstash encoder and promotes the MDC keys into top-level JSON fields. That structure is what makes the model's job tractable: it reads fields, not prose.

sample event	SERVICE=BACKEND, LEVEL=WARN
<pre>{ "@timestamp": "2026-08-19T14:22:07.914Z", "level": "WARN", "logger_name": "com.Market.MeatShop.Security.Services.AuthService", "message": "login field to Account with email admin@meat-shop.com , account type : EMPLOYEE", "requestId": "6f2a1c88-0d31-4a77-9a4e-1b2c3d4e5f60", "ip": "203.0.113.9", "method": "POST", "endpoint": "/auth/login", }</pre>	

```
"instanceId": "1",
"instanceName": "Main"
}
```

Signals already present in the application

The security module emits several events that map cleanly onto attacker behaviour, which is why the model is given the raw lines rather than a pre-filtered subset:

- `AuthService` logs a distinct warning for a wrong password against a known account and for an email that was never indexed. Repetition of the second across many addresses is enumeration; repetition of the first against one address is a brute force.
- `SessionService.traceSession` logs any access attempt against a session in `OBSERVED`, `CHALLENGED`, `STOLEN` or `REVOKED` state.
- `FingerPrintService` logs the full trust-score calculation, so a score that collapses from 95 to 20 between two requests on one session is visible as a stolen-token signal.
- `GlobalExceptionHandler` logs every `AuthorizationDeniedException`, which is what privilege probing against `/auth/roles` or `/employees` produces.

07 AI analysis server

Server 2 holds no business data. It has network access to exactly one thing on server 1, the Loki HTTP port, and it reaches it in both directions: reading log slices and writing verdicts. Ollama binds to loopback, so the model is not reachable from anywhere except the script that drives it.

RUNTIME

Ollama on `127.0.0.1:11434`, model `llama3` pulled once and kept warm

DRIVER

`/usr/local/bin/meatshop-analyse.sh`, invoked by cron every 5 minutes

STATE

`/var/lib/meatshop-ai/backend.timestamp` and `postgres.timestamp`, one nanosecond value each

MUTUAL EXCLUSION

`flock` on `/var/lock/meatshop-ai.lock`, non-blocking

EGRESS

Loki only. No log content leaves the two hosts, which is the reason for hosting the model rather than calling an API

The prompt contract

The model is asked for four fields and nothing else. Constraining the vocabulary of `attack_type` and `status` matters more than the wording of the instruction, because Grafana groups on those values and a free-text answer produces a table with one row per phrasing.

VERDICT SCHEMA PUSHED TO LOKI

FIELD	TYPE	CONSTRAINT AND USE
<code>risk_percentage</code>	integer	0 to 100, clamped after parsing. Grafana unwraps this into the gauge.
<code>attack_type</code>	enum	<code>none</code> , <code>brute_force</code> , <code>credential_stuffing</code> , <code>sql_injection</code> , <code>enumeration</code> , <code>session_hijack</code> , <code>dos</code> , <code>other</code>
<code>reason</code>	string	One sentence, newlines stripped, truncated at 400 characters so a runaway answer cannot break the stream.
<code>status</code>	enum	<code>CLEAR</code> <code>WATCH</code> <code>ALERT</code>

The driver script

Numbered comments in the source mark the eight stages: lock, window, pull, infer, sanitise, push, checkpoint, and the fixed ordering of the two sources.

`/usr/local/bin/meatshop-analyse.sh`

SERVER 2

```
#!/usr/bin/env bash
# /usr/local/bin/meatshop-analyse.sh
# Reads a bounded slice of Loki, asks Llama 3 what it means, pushes a verdict back.
set -Eeuo pipefail
```

```

LOKI="http://10.10.0.11:3100"
OLLAMA="http://127.0.0.1:11434"
MODEL="llama3"
STATE_DIR="/var/lib/meatshop-ai"
LOCK_FILE="/var/lock/meatshop-ai.lock"
BOOTSTRAP_WINDOW=600          # first run looks back 10 minutes
MAX_LINES=800                 # cap the prompt, newest lines win

mkdir -p "$STATE_DIR"
log() { printf '%s %s\n' "$(date -Is)" "$*" >&2; }

# --- 1. sequential lock -----
# Non-blocking. If the previous cycle is still inside Ollama, this one exits
# instead of queueing, so slow inference can never stack up overlapping runs.
exec 9>"$LOCK_FILE"
if ! flock -n 9; then
    log "another cycle holds the lock, skipping"
    exit 0
fi

# --- 2. window from the checkpoint -----
# start = the exact end of the previous window. Never "now minus N".
# That is what removes the gap: windows abut, they do not slide.
read_checkpoint() {
    local stamp="$STATE_DIR/$1.timestamp"
    if [[ -s "$stamp" ]]; then
        cat "$stamp"
    else
        echo $(( ($(date +%s) - BOOTSTRAP_WINDOW) * 1000000000 ))
    fi
}

analyse_source() {
    local name="$1" selector="$2" push_job="$3"
    local start end payload lines prompt raw verdict

    start="$(read_checkpoint "$name")"
    end="$(date +%s)000000000"

    if (( end <= start )); then
        log "$name: clock went backwards, refusing to query"
        return 0
    fi

    log "$name: window ${start} -> ${end}"

# --- 3. pull the slice -----
    payload="$(curl -sS --get "$LOKI/loki/api/v1/query_range" \

```

```
--data-urlencode "query=${selector}" \
--data-urlencode "start=${start}" \
--data-urlencode "end=${end}" \
--data-urlencode "limit=${MAX_LINES}" \
--data-urlencode "direction=backward")"
```

```
lines="$(jq -r ' [.data.result[]?.values[]?[1]] | reverse | .[]' <<<"$payload")"
```

```
if [[ -z "$lines" ]]; then
    log "$name: no lines in window, checkpoint advanced, no verdict pushed"
    printf '%s' "$end" > "$STATE_DIR/$name.timestamp"
    return 0
fi
```

```
# --- 4. inference, no timeout -----
# curl runs without --max-time on purpose. A 40 second answer and a
# 6 minute answer are both allowed to finish; the checkpoint is not
# written until the push below succeeds.
```

```
prompt="$(cat <<PROMPT
```

You are a security analyst reviewing \${name} logs from a Spring Boot and PostgreSQL deployment. Decide whether these lines show an attack in progress.

Answer with one JSON object and nothing else. Use exactly these keys:

```
risk_percentage    integer 0-100
attack_type        one of: none, brute_force, credential_stuffing,
                   sql_injection, enumeration, session_hijack, dos, other
reason            one sentence naming the specific evidence
status            one of: clear, watch, alert
```

Rules: repeated failures from one IP are brute_force. The same failure across many accounts is enumeration. SQL keywords or quote characters in request parameters are sql_injection. Routine traffic is risk 0, none, clear.

LOGS:

```
${lines}
```

```
PROMPT
```

```
)"
```

```
raw="$(jq -n --arg m "$MODEL" --arg p "$prompt" \
    '{model:$m, prompt:$p, stream:false, format:"json",
    options:{temperature:0, num_ctx:8192}}' \
    | curl -sS "$OLLAMA/api/generate" -H 'Content-Type: application/json' --data-bin
```

```
# --- 5. sanitise -----
# Ollama returns the model text inside .response. Even with format:json
# that text can arrive fenced, prefixed with prose, or truncated. Strip
# fences, keep the outermost brace pair, then force every field to the
# right type so Grafana's unwrap never meets a string.
```



```

verdict="$(jq -r '.response // empty' <<<"$raw" \
| sed -e 's/^[[:space:]]*```\(json\)\{0,1\}?' -e 's/```\[[:space:]]*$//'\ \
| tr -d '\000-\010\013\014\016-\037' \
| grep -o '{.*}' | head -n 1 \
| jq -c '{
  risk_percentage: ((.risk_percentage // 0) | tonumber? // 0 | floor
                    | if . < 0 then 0 elif . > 100 then 100 else . end),
  attack_type:      ((.attack_type // "none") | tostring | ascii_lowercase),
  reason:           ((.reason // "no reason returned") | tostring
                    | gsub("[\\n\\r]"; " ") | .[0:400]),
  status:           ((.status // "clear") | tostring | ascii_lowercase)
}' 2>/dev/null || true)"

if [[ -z "$verdict" ]]; then
  log "$name: model output unusable, checkpoint held"
  verdict='{ "risk_percentage":0, "attack_type":"other", "reason":"model returned unparse
fi

# --- 6. push the verdict back into Loki -----
jq -n --arg job "$push_job" --arg src "$name" \
  --arg ts "$(date +%s)000000000" --arg line "$verdict" \
  '{streams:[{stream:{job:$job, source:$src, application:"MeatShop"},
              values:[[$ts, $line]]}]}' \
| curl -sS -X POST "$LOKI/loki/api/v1/push" \
  -H 'Content-Type: application/json' --data-binary @- \
  --fail-with-body

# --- 7. checkpoint -----
# Written last, and set to the WINDOW END, not to the current time.
# Inference duration therefore never becomes an unread gap.
printf '%s' "$end" > "$STATE_DIR/$name.timestamp"
log "$name: verdict pushed, checkpoint = $end"
}

# --- 8. strict order under one lock -----
analyse_source backend '{service=~"backend|backend_2"}' meatshop_backend_ai_alerts
analyse_source postgres '{service="postgres"}' meatshop_postgres_ai_alerts

log "cycle complete"

```

Why the sanitiser is three stages rather than one

Ollama with `format:"json"` returns valid JSON in the envelope, but the model text inside `.response` is still generated text and fails in predictable ways. Each stage handles one of them:

- `sed` removes a leading ````json` fence and a trailing fence, which appear when the model treats the answer as a code block.
- `grep -o '{.*}' | head -n 1` discards any preamble or trailing commentary and keeps the first complete brace pair.
- The `jq` filter coerces types. `tonumber? // 0` survives `"85"` and `"eighty-five"` alike, `floor` removes a decimal, and the clamp keeps the gauge inside its own axis. Without this, one string value makes the LogQL `unwrap` drop the sample and the gauge silently reads stale.

FAILURE IS RECORDED, NOT SWALLOWED

If the output cannot be parsed at all, the script pushes a synthetic verdict with `status: watch` and a reason naming the parse failure. An operator looking at the dashboard sees that the analyser ran and could not read the answer, which is different from seeing nothing and assuming the system is quiet.

```
crontab -e
```

```
USER: OPS, SERVER 2
```

```
# Every 5 minutes. flock inside the script handles overlap, so the  
# schedule can stay fixed even when a cycle runs long.  
*/5 * * * * /usr/local/bin/meatshop-analyse.sh >> /var/log/meatshop-ai.log 2>&1
```

08 Cycle timing and the zero-gap guarantee

The naive way to schedule a log analyser is to query the last five minutes every five minutes. That works until inference runs long, the machine is busy, or cron drifts, at which point log lines fall into the seam between one window and the next and are never read by anything. The checkpoint file exists to make that impossible.

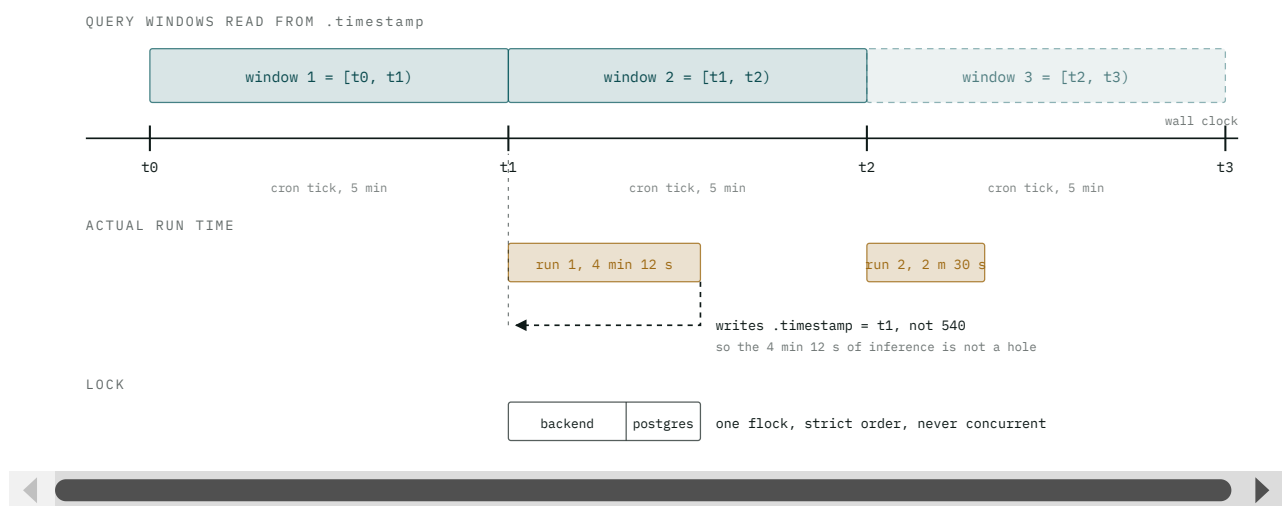


FIGURE 2 Window boundaries are shared, not recomputed. Because the checkpoint stores the query end rather than the completion time, a slow run shifts nothing: window 2 still begins exactly where window 1 stopped. The lock lane shows the two sources analysed back to back inside a single lock rather than in parallel.

What each mechanism removes

MECHANISM	FAILURE IT REMOVES	HOW
<code>.timestamp</code> checkpoint	Log lines written during a slow run are never analysed	The next window starts at the previous window's end, so run duration cannot open a gap.
Write-after-push ordering	A crashed or failed push silently marks the window as read	The checkpoint is written on the last line of the function. If the push fails, <code>set -e</code> aborts and the same window is retried next cycle. Duplicate analysis is acceptable; a missed window is not.
No <code>--max-time</code> on inference	A long answer is killed mid-generation and the window is lost	The request runs to completion. Overlap is handled by the lock, not by cutting inference short.
<code>flock -n</code>	Two cycles query Loki and drive Ollama at once	The second process finds the lock held and exits at once. Its window is not lost, because the checkpoint was never advanced, so the following successful run picks up a wider window covering both intervals.

MECHANISM	FAILURE IT REMOVES	HOW
Sequential source order	Two models loaded at once exhaust RAM on the analysis host	Backend analysis finishes before the PostgreSQL call begins. One model in memory at a time, and a deterministic order in the log.
Clock sanity check	An NTP step backwards produces an inverted range that Loki rejects	<code>end <= start</code> returns early without touching the checkpoint.

THE TRADE BEING MADE

Skipping a locked cycle rather than queueing it means the analysis interval is a floor, not a guarantee. Under sustained slow inference the effective cadence stretches and each window covers more time. That is the correct trade for this workload: a wider window analysed once beats a backlog of overlapping runs competing for the same GPU or CPU.

09 Threat simulation runbook

This is a full loop test. Each step generates traffic, confirms it reached Loki, drives the analyser, and checks that the verdict changed the dashboard. Run it against a staging deployment. Every address, token and credential below is a placeholder.

01 Simulate a brute force against login

Fifteen bad passwords against one account in a few seconds. This is what `AuthService` logs as a wrong-password warning and what the `meatshop-auth` jail counts.

attacker shell

BRUTE FORCE

```
for i in $(seq 1 15); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST https://api.meat-shop.com/auth/login \
    -H 'Content-Type: application/json' \
    -H 'X-Real-IP: 203.0.113.9' \
```

```
-d '{"email":"admin@meat-shop.com","password":"wrong- '$i'"}'
done
```

Expect a column of 403 responses. After eight, Fail2ban bans 203.0.113.9 and the rest never reach the application. Confirm the ban:

server 1

VERIFY BAN

```
fail2ban-client status meatshop-auth
# Banned IP list: 203.0.113.9
```

02

Simulate account enumeration

One password against many addresses. This produces the “not indexed” warning from AuthService and is the signal that separates enumeration from brute_force in the model's answer. Send from an ignored source so the jail does not stop the test early.

attacker shell

ENUMERATION

```
for u in ali sara omar lina admin root test; do
  curl -s -o /dev/null \
    -X POST https://api.meat-shop.com/auth/login \
    -H 'Content-Type: application/json' \
    -d '{"email":"'$u'@meat-shop.com","password":"Password123!"}'
done
```

03

Simulate an injection probe

MeatShop uses JPA and parameterised queries, so these do not reach the database as SQL. The point of the test is the log evidence: quote characters and SQL keywords in a request parameter, which is what the model is told to score as sql_injection, plus the PostgreSQL and validation errors they produce.

attacker shell

INJECTION STRINGS IN A FILTER PARAM

```
curl -s -G https://api.meat-shop.com/products/filter \
  -H 'Authorization: Bearer <token>' \
  --data-urlencode "productName=' OR '1'='1"

curl -s -G https://api.meat-shop.com/products/filter \
  -H 'Authorization: Bearer <token>' \
  --data-urlencode "productName=meat'; DROP TABLE products;--"
```

THIS ALSO TESTS AUTHORIZATION

Sending the same requests with no token exercises a second path.

`JwtFilter` lets an unauthenticated request continue, and the endpoint's `@PreAuthorize` rejects it, which `GlobalExceptionHandler` logs as an authorization denial. A burst of those is the privilege-probing signal from Section 06.

04

Confirm the traffic reached Loki

Before checking the AI, verify the raw evidence is queryable. Use Grafana's Explore view or the HTTP API directly.

server 1

LOGQL AGAINST LOKI

```
# login failures in the last 15 minutes, grouped by source IP
curl -sG http://localhost:3100/loki/api/v1/query_range \
  --data-urlencode 'query=sum by (ip) (count_over_time(
    {service=~"backend|backend_2"} | json
    | message =~ ".*login field.*" [15m]))' \
  --data-urlencode 'start="'$(date -d '15 min ago' +%s)'"'000000000
  | jq '.data.result'
```

05

Drive the analyser by hand

Do not wait for cron. Run the script directly on server 2 and read its own log. Because it is idempotent per window, a manual run followed by the scheduled one causes no harm.

server 2

MANUAL CYCLE

```
# optional: replay the last 15 minutes by rewinding the checkpoint
echo $(( ($(date +%s) - 900) * 1000000000 )) \
  | sudo tee /var/lib/meatshop-ai/backend.timestamp

sudo /usr/local/bin/meatshop-analyse.sh

# watch it work: window bounds, then the pushed verdict per source
tail -n 30 /var/log/meatshop-ai.log
```

A healthy run logs a window for `backend`, a pushed verdict, then the same for `postgres`, ending with `cycle complete`. If a second invocation overlaps the first, it logs `another cycle holds the lock, skipping` and exits zero.

06

Confirm the verdict landed

The analyser pushes to a job label, so the verdict is itself a Loki stream. Read it back the same way the dashboard does.

server 1

READ THE AI STREAM

```
curl -sG http://localhost:3100/loki/api/v1/query_range \
  --data-urlencode 'query={job="meatshop_backend_ai_alerts"} | json' \
  --data-urlencode 'start="$(date -d '10 min ago' +%s)"'000000000 \
  | jq '.data.result[].values[][1] | fromjson'
```

expected shape after the brute force test:

```
# { "risk_percentage": 82, "attack_type": "brute_force",
#   "reason": "15 failed logins for admin@meat-shop.com from 203.0
#   "status": "alert" }
```

07

Watch the dashboard update

Open the **AI Threat Detection Monitor** dashboard in Grafana. It refreshes every 10 seconds, so within two refresh cycles of the verdict landing:

- The **Current Risk Level** gauge for the backend row moves to the score the model returned. Its query is

```
max_over_time({job="meatshop_backend_ai_alerts"} | json |  
unwrap risk_percentage [$_range]))
```

, so it tracks the peak inside
the selected range.

- The **Threat History** table gains a row with the attack type, reason and status.
- The **Raw AI Payloads** logs panel shows the full JSON verdict for audit.

Set the dashboard range to the last 15 minutes so the injection and enumeration verdicts sit alongside the brute-force one and the gauge reflects the worst of the three.

08

Reset after the test

```
cleanup BOTH HOSTS  
  
# server 1: lift the test ban  
fail2ban-client set meatshop-auth unbanip 203.0.113.9  
  
# server 2: let checkpoints catch up to now so no replay of test data  
sudo rm -f /var/lib/meatshop-ai/backend.timestamp \  
          /var/lib/meatshop-ai/postgres.timestamp
```

Removing the checkpoints makes the next run bootstrap from a 10-minute look-back rather than replaying the whole test window. The verdicts already pushed stay in Loki until they age out at the 72-hour retention limit.

10 Known gaps and next steps

This detection layer sits on top of the running application without changing it, which is its main strength and also the source of its limits. A few things are worth stating plainly so they are tracked rather than assumed.

LATENCY

NO ACTIVE HEALTH CHECK

Detection is periodic, not live. An attack that completes inside one 5-minute window is scored after it finishes. Fail2ban is the real-time layer; the model is the interpreter.

Open-source Nginx removes an upstream only after live requests fail against it, and the compose file lacks an application healthcheck and a restart policy. Both are noted in Section 05.

CLIENT IP RESOLUTION

Without `forward-headers-strategy: NATIVE`, MDC `ip` logs the proxy address and the model loses its correlation key. This is the single highest-value fix in the report.

COMMITTED SECRET

The default JWT signing key ships in `application.yaml`. Rotate it and inject it only through the environment before this deployment faces real traffic.

MODEL DETERMINISM

Llama 3 at `temperature 0` is stable but not identical run to run. Treat the score as a priority signal, not a fact, and keep the raw payload for a human to read.

SINGLE LOKI NODE

Loki runs filesystem storage with a replication factor of 1. If server 1 is lost, so is the log history and the verdict stream. Off-host backup is the next infrastructure step.

THE SHAPE OF THE DESIGN

Two layers cover two timescales. Fail2ban acts in seconds on patterns it can match with a regex, which stops the loud and repetitive attacks at the network edge. Llama 3 acts in minutes on patterns that need reading rather than matching, and it does so without any log line leaving the two hosts. Neither layer depends on the other, so either can be tuned or replaced on its own.

MeatShop Threat Pipeline. Internal infrastructure and detection reference. Spring Boot 3.4.5 on Java 21, Nginx edge, PostgreSQL 17, Grafana Loki, Ollama with Llama 3. Configuration values are illustrative placeholders.

Prepared for the MeatShop platform team · Revision 1.0 ·
2026-08-19

Applies to backend image tag drawn from the repository
VERSION file.